SDN 및 P4 기반의 무손실 이동 표적 방어(MTD) 메커니즘에 관한 연구

A Study on Lossless Moving Target Defense (MTD) Mechanism Based on SDN and P4

이세영
Seyoung Lee

요약

본 연구에서는 네트워크 보안에서 문제가 되는 정적 IP 구성의 취약성을 해결하기 위해 P4 기반 SDN 환경에서의 무손실 MTD 메커니즘을 제안한다. MTD는 공격 표면을 동적으로 바꿔서 공격자의 정찰을 방해하는 방어 기술인데, 기존에 연구된 SDN 기반 MTD들은 컨트롤 평면에서 주소 변이를 처리하다 보니 컨트롤러 병목이나 패킷 유실 같은 문제가 있었다. 본 연구에서는 이런 문제를 해결하기 위해 P4 프로그래밍이 가능한 데이터 평면에서 직접 주소 변이를 수행하는 방식을 설계하였다. 이중 매핑 테이블과 플로우 추적 기반의 점진적 전환 알고리즘을 사용해서 기존 연결이 끊기지 않도록 하는 것이 핵심이다. 현재 Mininet과 BMv2로 테스트베드를 구축한 상태이고, P4 핵심 로직을 구현하는 중이다.

주제어: 이동 표적 방어 (MTD), 소프트웨어 정의 네트워킹 (SDN), P4, 무손실 주소 변이, 네트워크 보안, 데이터 평면

1. 서론

1.1 연구 배경 및 동기

현대 네트워크는 IP 주소, 포트, 라우팅 경로 같은 주요 자원이 고정되어 있는 정적 구조로 운영되고 있다. 이런 구조에서는 공격자가 시간을 충분히 갖고 정찰을 할 수 있기 때문에, 한 번 취약점이 발견되면 계속 공격에 활용된다. 쉽게 말해 방어자는 모든 곳을 다 막아야 하는데, 공격자는 하나만 뚫으면 되는 구조인 셈이다.

기존에는 방화벽이나 IPS, NAC 같은 수동적인 방어 체계를 사용해 왔는데, 제로데이 공격이나 APT 같은 지능형 위협에는 이런 방식으로 대응하기가 어려운 것이 현실이다. 이런 문제를 해결하기 위해 이동 표적 방어 (moving target defense, MTD) 기술이 주목받고 있다. MTD는 공격 표면을 동적으로 바꿔서 공격자의 정찰을 무력화하는 방어 전략이다.

그런데 MTD, 특히 네트워크 주소변이 기술을 기존의 고정 IP 기반 보안 체계에 적용하면 방화벽이나 NAC, IPS 같은 보안 시스템과 충돌하는 문제가 생길 수 있다. 그래서 실제로 적용하려면 해결해야 할 과제가 아직 남아 있다.

한편, SDN (Software-Defined Networking)은 네트워크의 제어 평면과 데이터 평면을 분리해서 중앙 집중적으로 네트워크를 관리할 수 있게 하는 기술이다. 초기 SDN에서는 OpenFlow라는 프로토콜을 통해 제어 평면을 프로그래밍하는 데 초점이 맞춰져 있었는데, 데이터 평면은 여전히

고정된 프로토콜 헤더에 의존하는 구조였다. 그러다 보니 OpenFlow의 매치 필드가 초기 12개에서 41개 이상으로 늘어나는 등 복잡성이 계속 증가하는 문제가 있었다. P4 (Programming Protocol-Independent Packet Processors)는 이런 SDN 데이터 평면의 한계를 극복하기 위해 등장한 고수준 프로그래밍 언어이다. P4 덕분에 제어 평면뿐만 아니라 데이터 평면까지 프로그래밍이 가능해져서, 진정한 의미의 SDN이 실현되었다고 볼 수 있다. P4를 쓰면 스위치 수준에서 패킷 파싱이나 헤더 조작, 매치-액션 처리를 직접 정의할 수 있기 때문에, 기존에 컨트롤러한테 맡겨야 했던 기능들을 데이터 평면에서 와이어 속도 (wire speed)로 처리할 수 있게 되었다. 이건 MTD에서 핵심인 주소 변이를 고속으로 처리할 수 있는 가능성을 열어주었다고 할 수 있다.

1.2 연구 목적 및 범위

본 연구의 목적은 P4로 확장된 SDN 환경에서 보안성 강화와 가용성 유지 사이의 트레이드오프를 해결하는 무손실 MTD 메커니즘을 설계하고 검증하는 것이다. 아직 구현과 실험이 완료되지 않은 단계이므로, 본 논문에서는 시스템 설계와 예상 결과를 중심으로 서술한다.

구체적으로는 다음과 같은 목표를 가지고 있다. 먼저 P4 프로그래밍을 활용해서 경계 스위치의 데이터 평면에서 직접 가상 IP-실제 IP 매핑을 수행하는 아키텍처를 설계한다. 이렇게 하면 SDN 컨트롤러의 개입을 최소화하고 주소 변이에 따른 지연을 마이크로초 수준으로 줄일 수 있다. 그 다음으로 주소 변이 과정에서 기존 TCP 세션이 끊기지 않도록 무손실 전환 알고리즘을 제안한다. 플로우 상태 추적 메커니즘과 이중 매핑 테이블을 사용한 점진적 전환 방식을 도입할 예정이다. 마지막으로 Mininet과 BMv2 소프트웨어 스위치로 프로토타입 실험 환경을 구축해서, 보안성이나 성능을 정량적으로 평가하고자 한다.

2. 관련 연구

본 연구를 진행하기에 앞서 MTD 기술 동향, SDN 및 P4 프로그래밍, 그리고 관련 보안 연구에 대한 문헌 조사를 수행하였다. 읽어본 논문 목록은 다음과 같다.

1) Jalowski 등: A Survey on Moving Target Defense for Networks: A Practical View
2) 이수원 등: 네트워크 보호체계에서 네트워크 주소변이 기술 적용에 대한 영향성 연구
3) Bosshart 등: P4: Programming Protocol-Independent Packet Processors
4) 조부승: P4 프로그래머블 네트워크를 통한 정책기반 인-네트워크 보안 관리 방법
5) Hyder와 Fatima: Towards Crossfire Distributed Denial of Service Attack Protection Using Intent-Based Moving Target Defense Over Software-Defined Networking
6) 김수연과 박태준: P4 기반 방화벽에 대한 연구
7) 이승운과 노병희: SDN 모델링 및 시뮬레이션을 위한 Mininet과 OPNET 간 성능 비교 분석

아래에서는 이 논문들의 내용을 정리하면서, 본 연구의 배경이 되는 내용을 설명하겠다.

2.1 MTD 기술 동향

MTD 기법은 적용 대상 (what to move, 무엇을 이동시킬 것인가), 동작 방식 (how to move, 어떻게 이동시킬 것인가), 적용 시점 (when to move, 언제 이동시킬 것인가)의 세 가지 기준으로 분류할 수 있다. 적용 대상으로는 IP 주소 및 포트 변경, 경로 변경, 호스트 변경 등 다양한 변이 대상이 있으며, 동작 방식으로는 대부분의 연구에서 랜덤 방식을 채택하고 있는 것으로 보인다. 정찰 공격, DoS/DDoS, 제로데이 공격, APT 등 다양한 공격 시나리오에 대한 방어 효과가 비교분석되어 있는데, 대부분의 연구가 시뮬레이션 기반으로 이루어져 있어 실제 대규모 네트워크 환경에서의 검증은 아직 부족한 상황이다.

주소변이 기술을 실제로 적용하려면 기존 보안 장비와의 호환성도 고려해야 한다. IPS의 경우 시그니처 기반 탐지 방식이 페이로드 분석 위주이므로 주소변이에 의한 영향이 미미하지만, 방화벽의 경우 변이되는 주소 대역 전체를 정책에 반영해야 하는 과제가 있다. 이 문제를 해결하기 위해 변이 전/후 주소를 매핑할 수 있는 대표 IP 개념과 시간별 변이 이력을 결합한 통합 관리 모델이 제시된 바 있다. 이 부분을 읽으면서 주소변이 기술이 단순히 IP만 바꾸면 되는 것이 아니라는 점을 알게 되었다.

한편, SDN 환경에서는 크로스파이어 공격이라는 지능형 DDoS 공격도 존재한다. 직접적인 타겟 대신 타겟 주변의 링크를 범람시켜 네트워크를 마비시키는 특성이 있어서 탐지가 매우 어렵다고 한다. 본 연구를 진행하면서 이러한 유형의 공격이 존재한다는 것 자체가 상당히 흥미로웠다. IBN (intention-based networking, 의도 기반 네트워킹)과 MTD를 결합하여 ONOS SDN 컨트롤러의 REST API와 DNS 포트 리다이렉션으로 트래픽을 가상 섀도우 호스트로 분산시키는 방어 방식이 제안되었는데, 빈번한 DNS 리다이렉션이 정상 사용자의 latency에 미치는 영향에 대한 분석이 다소 부족하고, 공격 규모가 매우 클 경우 가상 섀도우 호스트 자체가 자원 고갈 상태에 빠질 수 있는 가능성도 있어 보였다.

2.2 SDN과 P4의 관계

SDN은 네트워크의 제어 평면과 데이터 평면을 분리해서 중앙 집중적으로 제어할 수 있게 하는 기술이다. 초기 SDN에서는 OpenFlow를 통해 제어 평면을 프로그래밍하는 것이 핵심이었는데, 데이터 평면 쪽은 여전히 고정된 프로토콜 헤더에 의존하는 구조였다. OpenFlow의 매치 필드가 12개에서 41개 이상으로 늘어나면서 복잡성이 계속 증가하는 문제도 있었다.

P4는 이런 SDN 데이터 평면의 한계를 극복하기 위해 나온 프로그래밍 언어이다. P4는 SDN과 다른 기술이 아니라 SDN의 데이터 평면을 발전시킨 것이다. 기존 SDN은 제어 평면만 프로그래밍 가능했는데, P4가 나오면서 데이터 평면까지 프로그래밍할 수 있게 되어 진정한 의미의 SDN이 실현되었다고 볼 수 있다. P4의 핵심 구성 요소는 헤더 정의, 파서, 매치-액션 테이블, 액션, 제어 프로그램이고, 사용자가 패킷의 파싱 방법과 처리

로직을 직접 정의할 수 있다. P4의 핵심 목표는 현장 재구성 가능성, 프로토콜 독립성, 타겟 독립성이다.

P4를 보안 분야에 활용한 연구들도 있다. P4 데이터 평면에서 네트워크 내 모든 플로우를 실시간으로 모니터링하면서 SDN 제어기와 연동하여 DDoS 공격이나 IP 스푸핑 공격에 대응하는 방법이 제안되었다. 기존 OpenFlow 기반 SDN 스위치에서는 플로우 상태 모니터링이 안 돼서 동적인 사이버 공격에 대응하기 힘들었는데, P4 스위치에서는 네트워크 단에서 바로 보안 공격을 탐지하고 대응할 수 있다는 것이다. 이 내용이 본 연구에서 P4를 선택하게 된 가장 큰 계기가 되었다. 또한 P4 기반으로 방화벽을 구현해서 패킷 필터링 같은 보안 기능을 데이터 평면에서 직접 수행할 수 있다는 연구 결과도 있었다. DNS 패킷에만 국한되는 DPI 시스템의 한계 같은 게 아직 남아있긴 하지만, P4로 보안 기능을 구현하는 것 자체는 충분히 가능하다는 확신을 얻었다.

2.3 기존 MTD 기술의 한계

위의 논문들을 읽으면서 정리한 기존 SDN 기반 MTD의 한계는 크게 세 가지이다.

첫 번째는 컨트롤러 병목 문제이다. 기존의 OpenFlow 기반 SDN 스위치에서는 보안 위협 패킷이 들어오면 SDN 제어기한테 어떻게 처리할지 물어본 다음 그 결과를 다시 받아와야 해서 즉각적인 대응이 안 된다. 두 번째는 패킷 유실 문제인데, 이게 사실 본 연구를 시작하게 된 가장 큰 동기이다. 주소 변이가 발생하는 순간에 이전 매핑 정보로 전송 중이던 패킷이 새 플로우 규칙과 안 맞아서 드롭되는 문제가 생긴다. TCP 같은 연결 지향 프로토콜에서는 세션이 끊겨서 재연결이 필요하게 되고, 실시간 서비스에서는 이게 심각한 QoS 저하로 이어진다. 변이 과정에서의 서비스 중단이나 보안 정책 무력화가 우려된다는 지적도 있다. 세 번째는 플로우 테이블 확장성 문제이다. 주소 변이 주기가 짧아질수록 플로우 테이블에 설치해야 하는 규칙 수가 급증하는데, P4 기반 Tofino 칩의 경우 O(10MB) 수준 (규칙 수가 아무리 증가해도 이를 관리할 수 있는 테이블의 사이즈는 10MB 수준으로 한정됨)의 SRAM밖에 없어서 수백만 개의 플로우 정보를 다 저장하기 어렵다고 한다.

이러한 한계를 극복하기 위해 본 연구에서는 P4 데이터 평면에서 직접 주소 변이를 수행하면서, 플로우 추적 메커니즘과 이중 매핑 테이블을 통한 점진적 전환 알고리즘을 도입하여 무손실 보장을 제공하고자 한다.

3. 연구 방법

3.1 시스템 아키텍처 및 구성 요소

제안하는 시스템은 크게 SDN 컨트롤러, P4 경계 스위치 (edge switch), P4 코어 스위치 (core switch) 세 가지로 나누어 진다.

먼저 SDN 컨트롤러는 전체 MTD 정책을 관리하는 중앙 제어 장치인데, 변이 주기를 언제로 할지, 가상 IP 풀은 어떻게 관리할지, 가상IP-실IP 매핑 테이블은 어떻게 만들지 같은 것들을 결정한다. 컨트롤러는 정책만 정해주고 실제 패킷 단위의 주소 변환은 P4 스위치가 알아서 하도록 설계했다. SDN

제어기가 직접 패킷마다 처리하면 지연 시간이 너무 커지는 문제가 있기 때문이다.

그 다음 P4 경계 스위치는 내부 네트워크와 외부 네트워크의 경계에 있는 스위치이다. 여기가 사실상 MTD의 핵심이라고 할 수 있다. 하는 일은 간단하게 말하면 IP 주소를 바꿔치기하는 것인데, 외부에서 들어오는 패킷은 목적지의 가상 IP를 실제 IP로 바꾸고 (ingress translation), 반대로 내부에서 나가는 패킷은 출발지의 실제 IP를 가상 IP로 바꾼다 (egress translation). 이 과정을 P4로 프로그래밍해서 데이터 평면에서 와이어 속도로 처리할 수 있다.

마지막으로 P4 코어 스위치는 내부 네트워크에서 패킷 포워딩을 담당한다. 내부 호스트끼리는 실제 IP로 통신하니까 MTD의 주소 변이가 일어나도 영향을 안 받는다. 이건 기존 연구에서 제안된 대표 IP 개념과 비슷한 원리이다. 사실 코어 스위치 부분은 경계 스위치에 비하면 구현이 단순한 편이다.

3.2 P4 기반 데이터 평면 설계

P4 경계 스위치의 데이터 평면은 크게 파서, 인그레스 파이프라인, 이그레스 파이프라인, 디파서 순서로 패킷을 처리한다. 여기서는 각 단계가 무슨 역할을 하는지 설명하고자 한다.

먼저 파서 (parser)는 들어온 패킷에서 이더넷 헤더, IPv4 헤더, TCP/UDP 헤더를 차례로 파싱해서 필요한 필드를 뽑아내는 단계이다. P4에서 파서는 일종의 상태 머신으로 정의된다. 여기서 좋은 점은 P4가 프로토콜에 독립적이라서 필요하면 사용자가 직접 메타데이터 헤더를 정의해서 추가할 수도 있다는 것이다.

그 다음이 인그레스 파이프라인 (ingress pipeline)인데, 여기가 제일 복잡하다. 매치-액션 테이블이 여러 개 있어서 패킷이 들어오면 순서대로 적용된다. 제일 먼저 가상IP-to-실IP 변환 테이블을 거치는데, 외부에서 온 패킷의 목적지 가상 IP를 실제 IP로 바꿔주는 역할을 한다. 이 테이블은 SDN 컨트롤러가 주기적으로 업데이트해주고, 현재 매핑과 이전 매핑을 동시에 갖고 있는 이중 매핑 구조이다. 왜 이전 매핑도 같이 들고 있어야 하는지는 3.3절의 무손실 알고리즘에서 설명하려고 한다. 그 다음에 플로우 상태 테이블이 있는데, P4 레지스터를 이용해서 지금 활성화된 플로우의 5-tuple 정보 (출발지 IP, 목적지 IP, 출발지 포트, 목적지 포트, 프로토콜)와 그 플로우가 어떤 매핑 버전을 쓰고 있는지를 기록해둔다. 마지막으로 포워딩 테이블에서 변환된 실제 IP를 보고 어느 포트로 보낼지 결정한다.

이그레스 파이프라인 (egress pipeline)은 인그레스와 반대 방향이다. 내부에서 외부로 나가는 패킷의 출발지 실제 IP를 가상 IP로 바꾼다. 근데 여기서 좀 신경 써야 할 부분이 있는데, 응답 패킷의 경우에는 아무 가상 IP나 쓰면 안 되고, 원래 요청이 들어왔을 때 사용된 가상 IP와 같은 걸 써야 한다. 안 그러면 외부 클라이언트 입장에서 전혀 다른 IP에서 응답이 온 것처럼 보이게 되기 때문이다. 그래서 플로우의 매핑 버전을 참조하는 과정이 필요하다.

마지막으로 디파서 (deparser)에서는 앞에서 수정한 헤더 필드들을 다시 패킷으로 조립하고, IP 헤더가 바뀌었으니까 IPv4 체크섬도 다시 계산해줘야 한다. 여기서 증분 체크섬 계산이라는 걸 쓰는데, 전체를 다시 계산하는 것보다 바뀐 부분만 반영하는 방식이라 오버헤드가 적다.

3.3 무손실 IP 변이 알고리즘

무손실 주소 변이에서 제일 중요한 건 매핑을 전환할 때 기존에 진행 중인 플로우가 끊기지 않게 하는 것이다. 이 부분이 사실 본 연구에서 가장 중요하면서도 어려운 부분이라고 생각한다. 단순하게 생각하면 매핑을 새 걸로 바꿔치기하면 될 것 같지만, 그렇게 하면 바꾸는 그 순간에 진행 중이던 패킷들이 드롭된다. 그래서 이중 매핑 테이블과 플로우 추적을 기반으로 한 점진적 전환 알고리즘을 제안한다.

알고리즘은 새 매핑 준비, 병행 운영, 그리고 정리의 3단계로 되어 있다.

새 매핑 준비: 일단 SDN 컨트롤러가 새 가상IP-실IP 매핑을 만들어서 P4 스위치에 보내는데, 기존 매핑을 덮어쓰는 게 아니라 이중 매핑 테이블의 비활성 슬롯에 따로 설치한다. 그러니까 테이블에 슬롯이 두 개 있어서 하나는 현재 쓰고 있는 매핑, 하나는 새로 넣는 매핑이 들어가는 구조이다. 이 단계에서는 기존 트래픽은 아무 영향 없이 현재 매핑을 그대로 쓴다.

병행 운영: 이 단계가 핵심이라고 할 수 있다. 새 매핑이 활성화되면 새로 들어오는 연결은 새 매핑을 사용한다. 근데 기존에 이미 진행 중이던 플로우는 어떻게 하느냐가 문제인데, 플로우 상태 테이블을 참조해서 이전 매핑을 계속 적용해주는 것이다. P4 레지스터에 각 플로우가 어떤 매핑 버전을 쓰고 있는지 기록해뒀기 때문에, 패킷이 들어올 때마다 그 정보를 보고 적절한 매핑을 골라서 적용할 수 있다. SRAM 레지스터 기반이라 패킷 당 라인 속도로 처리가 가능하다.

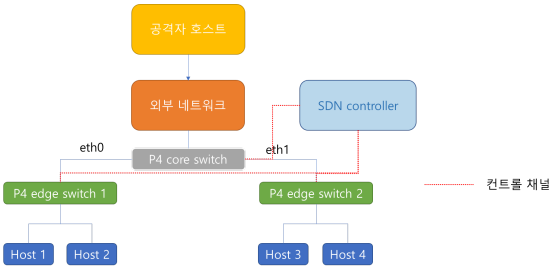
정리: 시간이 지나면 이전 매핑을 쓰는 플로우들이 하나둘씩 끝나게 된다. P4 카운터로 이전 매핑을 참조하는 플로우가 몇 개 남았는지를 계속 세고 있다가, 0이 되면 이전 매핑 슬롯을 해제하고 컨트롤러에 완료를 알린다. 플로우가 자연스럽게 끝나지 않는 경우를 대비해서 타임아웃도 설정해둔다.

정리하면 이 알고리즘의 핵심은, 매핑을 한 번에 확 바꾸는 게 아니라 점진적으로 전환하는 것이다. 그래서 이론적으로는 어떤 시점에서도 활성 플로우의 패킷이 매핑 불일치로 드롭되는 일이 없어야 한다. 다만 실제 구현에서는 레지스터 경합이나 타이밍 이슈 같은 게 생길 수도 있을 것 같아서, 이 부분은 구현하면서 좀 더 면밀히 검증해 볼 생각이다.

3.4 실험 환경 구축

실험 환경을 구축하기 위해 Mininet 에뮬레이터 (https://github.com/mininet/mininet) 와 BMv2 P4 소프트웨어 스위치를 사용하기로 했다. Mininet 소스코드가 최종 업데이트 된 시점이 3년이 넘는다는 점이 의구심이 남지만, 다른

연구에서도 Mininet과 ONOS 컨트롤러로 SDN 기반 MTD의 성능을 검증한 사례가 있어서 이를 그대로 채택하기로 판단했다. Mininet과 OPNET을 비교한 연구에 따르면 Mininet은 처리량 측면에서는 실제 환경을 잘 반영한다고 한다. 다만 지연 시간이나 지터 같은 건 정밀도가 떨어질 수 있다고 하니까 나중에 결과를 해석할 때 이 점은 감안해야 할 것으로 보인다.



[도표 1. 실험 환경 토폴로지 요약]

토폴로지는 도표 1에서 보듯 외부 네트워크에 공격자 호스트를 하나 두고, P4 경계 스위치 2대, P4 코어 스위치 1대, 그리고 내부에 서버 호스트 4대를 배치하는 구성으로 했다. SDN 컨트롤러는 아직 ONOS를 쓸지 Ryu를 쓸지 확정하지 못했는데, P4Runtime API를 통해 통신하는 건 동일하다. 실험할 때 가상 IP 풀은 /16 서브넷으로 65,536개 주소를 쓰고, 변이 주기는 5초에서 60초까지 바꿔가면서 테스트할 예정이다. TCP 연결은 iperf3 (https://github.com/esnet/iperf) 로 장기 세션을 만들 생각이다. 비교 대상은 MTD를 아예 안 쓴 환경, OpenFlow 기반으로 컨트롤러에서 MTD를 처리하는 환경, 그리고 본 연구에서 제안하는 P4 기반 무손실 MTD 이렇게 세 가지이다.

현재 Mininet이랑 BMv2로 테스트베드는 구축을 마치고, ping 테스트 등으로 기본적인 연결성은 확인한 상태이다. P4 핵심 로직은 아직 구현하는 중인데, P4 문법이 C랑 비슷한 것 같으면서도 다른 부분이 있어서 익숙해지는 데 생각보다 시간이 걸리고 있다.

4. 연구 결과

차후 핵심 로직 구현 등을 마친 후 다음과 같은 두 부문에서 테스트 및 분석을 진행할 예정이다.

4.1 예상되는 보안성 분석

향후 스캔 공격 시나리오를 구성해서 MTD의 보안 효과를 평가할 예정이다. nmap이 네트워크 스캔 분야에서 사실상의 표준이므로 이를 채택할 예정이다. 공격자가 외부 네트워크에서 대상 서브넷에 TCP SYN 스캔, UDP 스캔, 핑 스윕을 수행하는 상황을 만들고, MTD 적용 시 실제 서비스 호스트의 탐지 성공률을 측정하려고 한다.

MTD의 보안 효과는 공격자의 정찰 성공 확률로 정량화할 수 있다. IP 주소 및 포트 변경을 통한 MTD가 정찰 공격에 대해 가장 직접적인 방어 효과를 가지는 것으로 확인된 바 있다. 가상 IP 풀의 크기를 N, 실제 호스트 수를 M, 변이 주기를 T, 공격자의 스캔 속도를 S (패킷/초)라 하면, 공격자가 변이 주기 내에 특정 호스트를 탐지할 확률은 1 - (1 - M/N)^(S*T)로 계산할 수 있다.

본 연구 설정 (N=65,536, M=4)에서 변이 주기 T가 짧아질수록 탐지 확률이 지수 함수적으로 감소할 것으로 예상되는데, 이 수식이 실제 환경에서도

유효한지는 실험을 통해 확인해 봐야 할 것 같다. 예를 들어 공격자의 스캔 속도가 1,000 패킷/초이고 변이 주기가 10초면 탐지 확률은 약 45.6%인데, 변이 주기를 5초로 줄이면 약 26.3%로 감소한다.

4.2 예상되는 네트워크 오버헤드 분석

네트워크 성능 평가를 위해 다음과 같은 지표들을 측정해 볼 계획이다.

지연 시간의 경우, P4 데이터 평면에서의 주소 변환에 소요되는 추가 지연을 측정할 것이다. P4 매치-액션 파이프라인의 각 스테이지는 ALU로 구현되어 하드웨어 속도로 처리가 가능하므로 추가 지연은 마이크로초 이내 수준으로 예상된다. 기존 OpenFlow 기반 MTD는 컨트롤러 왕복 시간이 추가돼서 밀리초 수준이었던 것과 비교하면 상당한 차이이다.

처리량은 iperf3를 통해 MTD 적용 전후의 TCP/UDP 처리량을 비교할 예정이다. Mininet이 처리량 측면에서 실제 환경을 잘 반영한다고 하니까 어느 정도 신뢰할 수 있을 것으로 본다. P4 스위치의 매치-액션 처리가 기본 포워딩과 비슷한 파이프라인 구조이기 때문에 처리량 저하는 크지 않을 것으로 예상한다.

패킷 유실률은 주소 변이 시점에서 패킷이 실제로 유실되는지 측정한다. 이중 매핑 테이블과 플로우 추적 메커니즘 덕분에 0%에 가까울 것으로 기대하고 있는데, 기존 OpenFlow 기반 MTD에서 변이 때마다 패킷이 유실되던 것과 비교해 볼 것이다. 세션 연속성도 확인할 건데, 장시간 TCP 세션 (HTTP, SSH 등)을 유지한 상태에서 주소 변이를 여러 번 수행하면서 세션이 끊기는지를 기록할 예정이다.

5. 결론

본 연구에서는 P4로 확장된 SDN 환경에서 무손실 MTD 메커니즘을 제안하였다. 기존 SDN 기반 MTD에서 문제가 되었던 컨트롤러 병목, 패킷 유실, 세션 단절을 P4 데이터 평면을 활용해서 해결하는 접근법을 제시하였고, 이중 매핑 테이블과 플로우 상태 추적 기반의 점진적 전환 알고리즘을 설계하였다.

현재까지 한 것을 정리하면, 문헌 분석을 통해 기존 MTD 기술의 한계와 P4 기반 접근법의 타당성을 확인하였고, 3계층 아키텍처와 무손실 IP 변이 알고리즘을 설계하였으며, Mininet과 BMv2 기반 테스트베드를 구축하고 P4 핵심 로직 구현에 착수하였다. 향후에는 P4 기반 주소 변이 및 무손실 로직 구현을 완료하고, Nmap 기반 스캔 공격 시뮬레이션과 iperf3 기반 가용성 측정을 수행할 예정이다. 이후 네트워크 오버헤드 및 보안성을 정량적으로 평가하고 최종 보고서를 완성할 계획이다.

장기적으로는 하드웨어 P4 스위치에서의 검증이 가장 시급한 과제라고 생각한다. 소프트웨어 스위치에서 아무리 좋은 결과가 나와도 실제 하드웨어에서 검증하지 않으면 실용성을 주장하기가 어렵기 때문이다. 아직 부족한 점이 많지만, 본 연구의 결과가 실제 네트워크

환경에서의 능동적 보안 체계 구축에 조금이나마 기여할 수 있기를 기대한다.

참고문헌

1. Jalowski, Ł., Zmuda, M., & Rawski, M. (2022). A Survey on Moving Target Defense for Networks: A Practical View. *Electronics, 11*(18), 2886.

2. 이수원, 황세영, 홍석규. (2023). 네트워크 보호체계에서 네트워크 주소변이 기술 적용에 대한 영향성 연구. *정보보호학회논문지, 33*(6), 939-946.

3. Bosshart, P., Daly, D., Gibb, G., Izzard, M., McKeown, N., Rexford, J., Schlesinger, C., Talayco, D., Vahdat, A., Varghese, G., & Walker, D. (2014). P4: Programming Protocol-Independent Packet Processors. *ACM SIGCOMM Computer Communication Review, 44*(3), 87-95.

4. 조부승. (2020). P4 프로그래머블 네트워크를 통한 정책기반 인-네트워크 보안 관리 방법. *융합보안논문지, 20*(5), 3-10.

5. Hyder, M. F., & Fatima, T. (2021). Towards Crossfire Distributed Denial of Service Attack Protection Using Intent-Based Moving Target Defense Over Software-Defined Networking. *IEEE.* https://ieeexplore.ieee.org/document/9509535

6. 김수연, 박태준. (2024). P4 기반 방화벽에 대한 연구. *한국통신학회 학술대회논문집,* 2024년 동계종합학술발표회.

7. 이승운, 노병희. (2018). SDN 모델링 및 시뮬레이션을 위한 Mininet과 OPNET 간 성능 비교 분석. *한국차세대컴퓨팅학회 논문지, 14*(2), 82-93.

8. Mininet (https://mininet.org/)

9. BMV2 Github repository (https://github.com/p4lang/behavioral-model)